

OCR Pipeline — Low-Level Design

Module ID: ocr **Owner:** TBD **Status:** Draft for Meeting 4 **PRD reference:** PRD §8.2 **Last updated:** 2026-05-31

1. Purpose & Scope

The OCR Pipeline extracts Hebrew and English text from chat screenshot images so that the same frontline classifier (`server/app/classifier.py`) can classify image content without a separate vision model. It is the first stage of the `POST /classify-image` endpoint and the reason Architecture A (Vision LLM) was not needed — text extracted from screenshots flows through the identical DictaBERT path.

Scope (in): - Input validation (size, MIME type) - PIL-based image preprocessing (resize, grayscale, adaptive threshold) - Tesseract OCR via `pytesseract` with `heb+eng` language packs - Structured output schema consumed by the classifier - Signal propagation for `image_unreadable` cases - Integration with `server/app/image_backends/strategies.py` strategy router

Scope (out): - Vision LLM classification of image content (Architecture A — out of scope per PRD §11) - On-device OCR (MLKit future work) - Cloud OCR services (breaks local-first privacy guarantee)

System context: see [architecture_diagrams.md](#) — this module is the "Tesseract OCR (heb+eng)" node inside the HomeNet boundary.

2. Public Interface (API Contract / Protocol)

2.1 Protocol definition

```
# server/app/image_backends/base.py (already exists – design enforces this shape)
from typing import Protocol, runtime_checkable

@runtime_checkable
class ImageProcessor(Protocol):
    async def process(
        self,
        image_bytes: bytes,
        content_type: str | None = None,
    ) -> "OcrResult":
        ...
```

2.2 OcrResult output schema

```
# server/app/image_backends/schemas.py (new file)
from dataclasses import dataclass
```

```

@dataclass(frozen=True)
class OcrResult:
    extracted_text: str          # UTF-8, stripped; empty string when no text detected
    confidence: float            # Mean Tesseract word confidence, 0.0-1.0
    lang_detected: list[str]     # subset of ["heb", "eng"] – scripts actually found
    bbox_count: int              # number of detected text bounding boxes
    image_unreadable: bool       # True when OCR engine returned nothing meaningful
    backend: str                 # always "ocr" for this module
    strategy: str                # propagated from the strategy router

```

`OcrResult` is consumed by `OcrBackend.process()` internally. The strategy router (`strategies.py`) wraps it in `ImageClassifyResult` (existing frozen dataclass in `base.py`) before returning to the FastAPI endpoint. The endpoint never sees `OcrResult` directly — it sees `ImageClassifyResult` .

2.3 Input contract

Field	Type	Constraint	Error response
<code>image_bytes</code>	<code>bytes</code>	≤ 4 MB	HTTP 413 from Gatekeeper middleware
<code>content_type</code>	<code>str</code> <code>None</code>	<code>image/jpeg</code> or <code>image/png</code>	HTTP 415 if present and wrong; missing = tolerated

Gatekeeper enforces the 4 MB ceiling (PRD §8.7); `OcrBackend` itself trusts the bytes are already validated and opens them with `PIL.Image.open()` .

2.5 Interface boundary & isolation guarantees

The Port (Protocol): `OcrBackend` — the ONLY symbol the server core imports from this module. The strategy router and the `/classify-image` handler depend on `OcrBackend` , never on `TesseractOcrBackend` or `EasyOcrBackend` directly.

```

# server/app/ocr/protocol.py
from typing import Protocol, runtime_checkable

@runtime_checkable
class OcrBackend(Protocol):
    async def process(
        self,
        image_bytes: bytes,
        content_type: str | None = None,
    ) -> "OcrResult":
        """Extract text from an image. Never raises on unreadable input –
        returns OcrResult(image_unreadable=True) instead.
        Must return within the configured OCR_TIMEOUT_S budget."""
        ...

```

`OcrResult` (see §2.2) is part of the port. Both the schema AND the contract that `process()` never raises on unreadable input are invariants every adapter must honour.

Concrete adapters that satisfy this Protocol:

Adapter	When to use	Lines to change to enable
<code>TesseractOcrBackend</code>	Default — MVP / Meeting 5+; local CPU, Hebrew + English language packs	(default — already wired in <code>lifespan()</code>)
<code>EasyOcrBackend</code>	Risk-mitigation per PRD §12: if Tesseract CER > 15% on real chat screenshots	one line in <code>main.py</code> <code>lifespan()</code> ; add <code>easyocr</code> to <code>requirements.txt</code>
<code>MlkitOcrBackend</code>	Future on-device OCR (Android side) — moves OCR off the server entirely	one line + new SDK contract; deferred to Phase 9
<code>StubOcrBackend</code>	Unit and contract tests; returns a fixed <code>OcrResult</code> from a fixture file	injected by the test fixture, not by <code>main.py</code>

Isolation rules (what this module MAY and MUST NOT touch): - May import: `stdlib`, `pytesseract`, `PIL` / `Pillow`, `numpy`, `cv2` (`opencv-python-headless`), `structlog`, `prometheus_client`, this module's own settings. - MUST NOT import: any concrete class from another module (only Protocols — and the OCR module is a leaf, so it imports no other Protocols at all). - MUST NOT import: `server.app.classifier.*` (the OCR module is text-extraction-only; the *strategy router* calls the classifier, not the backend). - MUST NOT import: `server.app.main` or anything in the composition root.

Contract test: `tests/contracts/test_ocr_backend_contract.py` — every adapter is parametrized through this suite. Fixtures provide: a Hebrew chat screenshot, a mixed-language screenshot, a blank image, and a corrupt JPEG. The suite asserts: (a) result schema matches `OcrResult` exactly, (b) `image_unreadable=True` is set on blank input (not an exception), (c) p99 latency over 50 runs is within the adapter's documented budget (Tesseract ≤ 2 s; EasyOCR ≤ 3 s), (d) `confidence` $\in [0.0, 1.0]$, (e) `lang_detected` is a subset of `["heb", "eng"]`.

Swap demo — Tesseract → EasyOCR (PRD §12 risk row):

```
# Before — server/app/main.py lifespan()
ocr: OcrBackend = TesseractOcrBackend(settings.ocr)

# After
ocr: OcrBackend = EasyOcrBackend(settings.ocr)
```

The strategy router (`OcrOnly` / `Pipeline` / `Parallel`), the `/classify-image` route handler, the audit log, and the metrics emitter all keep working unchanged — they only know the `OcrBackend` Protocol.

3. Internal Design

3.1 Package layout

```
server/app/image_backends/
├─ base.py          # ImageProcessor ABC + ImageClassifyResult (exists)
├─ stub.py          # Phase 1 no-op backend (exists)
├─ vision.py        # Vision LLM backend (exists, out of scope here)
├─ strategies.py    # Strategy router – OcrOnly / Pipeline / Parallel (exists)
├─ ocr.py           # OcrBackend – THE subject of this LLD (exists, will be refactored)
└─ schemas.py       # NEW: OcrResult dataclass + preprocessing params
```

3.2 Key classes

OcrBackend (refactored from `server/app/image_backends/ocr.py`)

```
class OcrBackend(ImageProcessor):
    """
    Full pipeline: validate → preprocess → OCR → emit OcrResult.
    The strategy router decides whether to pass the extracted text
    to the classifier; OcrBackend only extracts.
    """
    def __init__(
        self,
        tesseract_cmd: str | None = None,
        lang: str = "heb+eng",
        dpi: int = 300,
        threshold_block_size: int = 31,
        threshold_c: int = 10,
    ): ...

    async def process(
        self, image_bytes: bytes, content_type: str | None = None
    ) -> OcrResult: ...

    def _preprocess(self, image_bytes: bytes) -> "PIL.Image": ...
    def _run_tesseract(self, img: "PIL.Image") -> tuple[str, float, list[str], int]: ...
```

Current `ocr.py` calls `classify_text()` directly — this will be decoupled. The `OcrBackend` becomes extraction-only; the strategy router (or the caller) is responsible for passing extracted text to `classify_text()`. This separation makes the module testable in isolation.

ImagePreprocessor (extracted helper, lives in `ocr.py`)

```
class ImagePreprocessor:
    """Stateless preprocessing: resize → grayscale → adaptive threshold."""

    TARGET_HEIGHT = 1200    # pixels; maintains aspect ratio
```

```

MIN_HEIGHT = 400          # skip resize if already small enough

@staticmethod
def run(img: PIL.Image, block_size: int, c_value: int) -> PIL.Image:
    img = ImagePreprocessor._resize(img)
    img = img.convert("L")          # grayscale
    arr = np.array(img)
    thresh = cv2.adaptiveThreshold(
        arr, 255,
        cv2.ADAPTIVE_THRESH_GAUSSIAN_C,
        cv2.THRESH_BINARY,
        block_size,      # default 31 – must be odd
        c_value,         # default 10
    )
    return PIL.Image.fromarray(thresh)

```

Why these preprocessing params: - `TARGET_HEIGHT = 1200 px` — Tesseract accuracy degrades sharply below 150 DPI effective; most chat screenshots at phone resolution (1080p) need no upscaling. This cap prevents oversized inputs from slowing OCR. - `ADAPTIVE_THRESH_GAUSSIAN_C, block_size=31, c=10` — handles the mixed-background chat bubbles (light + dark themes) common in WhatsApp/Instagram screenshots. Global threshold fails on gradient backgrounds; adaptive per-region threshold handles both. - Grayscale before threshold — reduces color noise without affecting character shape.

TesseractRunner (thin wrapper, lives in `ocr.py`)

```

class TesseractRunner:
    TESSERACT_CONFIG = "--psm 6 -c preserve_interword_spaces=1"
    # psm 6: assume uniform block of text – optimal for chat screenshot regions
    # preserve_interword_spaces: critical for Hebrew RTL word segmentation

    def run(self, img: PIL.Image, lang: str) -> tuple[str, float, list[str], int]:
        """Returns (text, mean_confidence, lang_list, bbox_count)."""
        data = pytesseract.image_to_data(
            img, lang=lang, config=self.TESSERACT_CONFIG,
            output_type=pytesseract.Output.DICT,
        )
        # Filter low-confidence words (conf < 0)
        words = [w for w, c in zip(data["text"], data["conf"]) if c >= 0 and w.strip()]
        confs = [c for c in data["conf"] if c >= 0]
        text = " ".join(words).strip()
        mean_conf = sum(confs) / len(confs) / 100.0 if confs else 0.0
        langs = _detect_scripts(text)
        return text, mean_conf, langs, len(words)

```

Tesseract config rationale: - `--psm 6` (Assume a single uniform block of text): better than `--psm 3` (auto) for cropped chat screenshot regions; auto-OSD adds latency and misdetects RTL chat bubbles as tables. -

`preserve_interword_spaces=1` : Hebrew tokenization is space-separated — losing spaces conflates tokens and destroys word meaning. - `heb+eng` language pack combination: Israeli teen chat uses frequent code-switching

(Hebrew/English in the same message); running both simultaneously outperforms single-lang Tesseract on mixed text.

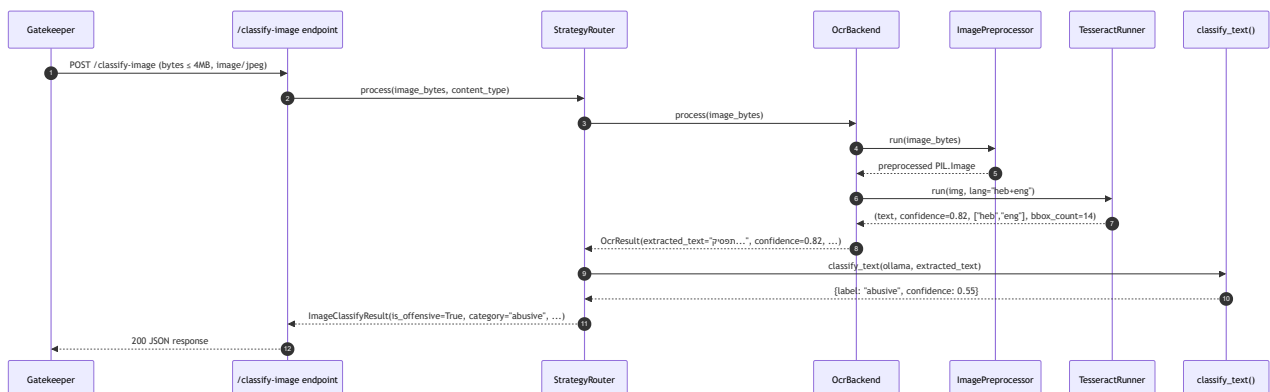
3.3 Extraction seam (modular monolith → standalone service)

`OcrBackend` depends only on `base.py` (the `ImageProcessor` ABC) and standard libraries (`pytesseract`, `PIL`, `numpy`, `cv2`). It has zero imports from `classifier.py`, `ollama_client.py`, or any other server module after the planned refactor. This clean boundary means:

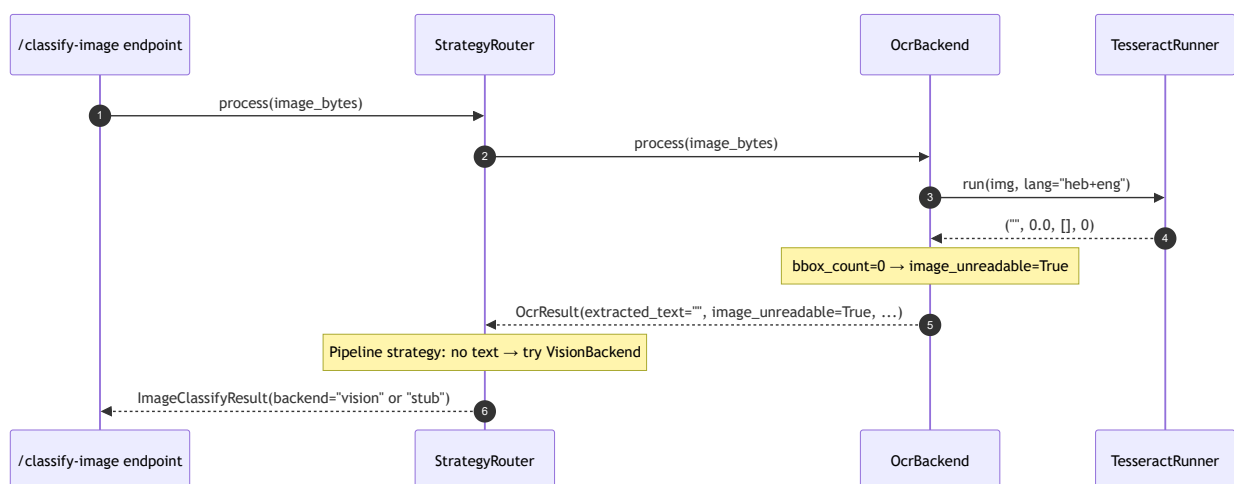
- **Test in isolation:** `pytest server/tests/test_ocr.py` with mock bytes, no Ollama.
- **Extract to microservice:** wrap `OcrBackend.process()` in a FastAPI endpoint (`POST /ocr`), deploy behind the Gatekeeper. The strategy router's `OcrOnly` strategy just changes its HTTP call target. Zero changes to `classifier.py`.

4. Sequence Diagrams

4.1 Happy path — OCR extracts text



4.2 Failure path — image unreadable



5. Data Model

5.1 Input → Output flow

image_bytes (raw JPEG/PNG)

→ ImagePreprocessor.run()

→ TesseractRunner.run()

→ OcrResult

→ classify_text()

→ ImageClassifyResult

→ PIL.Image (grayscale, thresholded)

→ (text: str, confidence: float,

lang_detected: list[str], bbox_count: int)

→ (all fields above + image_unreadable: bool)

→ {label, confidence, model_version, latency_ms}

→ (is_offensive, category, confidence,

extracted_text, backend, strategy)

5.2 image_unreadable signal semantics

Condition	extracted_text	image_unreadable	Router action
Text found, confidence ≥ 0.3	non-empty	False	Classify text
No bboxes returned	""	True	Pipeline: try VisionBackend; OcrOnly: return {category: "no_text"}
Tesseract binary not found	n/a	raises TesseractNotFoundError	Caught by _try_ocr() in strategies.py → degrade to vision
PIL cannot decode image	n/a	raises UnidentifiedImageError	Propagated as HTTP 422

6. Observability

6.1 Logger

Module logger: shomer.ocr via structlog.

```
import structlog
logger = structlog.get_logger("shomer.ocr")
```

Three example log lines (JSON-structured):

```
{"event": "ocr_success", "trace_id": "abc123", "module": "ocr",
 "latency_ms": 312, "bbox_count": 14, "confidence": 0.82,
 "lang_detected": ["heb", "eng"], "text_length": 87}

{"event": "ocr_no_text", "trace_id": "def456", "module": "ocr",
 "latency_ms": 198, "bbox_count": 0, "confidence": 0.0,
 "image_unreadable": true, "content_type": "image/jpeg"}
```

```
{
  "event": "ocr_tesseract_unavailable",
  "trace_id": "ghi789",
  "module": "ocr",
  "error": "TesseractNotFoundError",
  "fallback": "vision",
  "tesseract_cmd": "C:\\Program Files\\Tesseract-OCR\\tesseract.exe"
}
```

Fields on every OCR log line: `trace_id` , `module="ocr"` , `event` , `latency_ms` . Additional context per event type as shown above.

6.2 Config

`OcrSettings` (Pydantic-settings, loaded from environment):

Name	Type	Default	Env var	Description	Secret?
<code>tesseract_cmd</code>	<code>str</code> <code>None</code>	<code>None</code>	<code>TESSERACT_CMD</code>	Absolute path to <code>tesseract.exe</code> ; <code>None</code> = use PATH	No
<code>tesseract_lang</code>	<code>str</code>	<code>"heb+eng"</code>	<code>TESSERACT_LANG</code>	Language pack(s) passed to Tesseract <code>-l</code> flag	No
<code>ocr_dpi</code>	<code>int</code>	<code>300</code>	<code>OCR_DPI</code>	Target DPI for resize before OCR	No
<code>ocr_threshold_block_size</code>	<code>int</code>	<code>31</code>	<code>OCR_THRESHOLD_BLOCK_SIZE</code>	Adaptive threshold block size (must be odd)	No
<code>ocr_threshold_c</code>	<code>int</code>	<code>10</code>	<code>OCR_THRESHOLD_C</code>	Adaptive threshold constant C	No
<code>ocr_min_confidence</code>	<code>float</code>	<code>0.30</code>	<code>OCR_MIN_CONFIDENCE</code>	Below this mean word confidence, treat as no_text	No
<code>ocr_max_image_bytes</code>	<code>int</code>	<code>4_194_304</code>	<code>OCR_MAX_IMAGE_BYTES</code>	4 MB hard limit (backup to Gatekeeper)	No

```
# server/app/config.py (add to existing config module)
from pydantic_settings import BaseSettings
```

```
class OcrSettings(BaseSettings):
    tesseract_cmd: str | None = None
    tesseract_lang: str = "heb+eng"
    ocr_dpi: int = 300
    ocr_threshold_block_size: int = 31
    ocr_threshold_c: int = 10
    ocr_min_confidence: float = 0.30
```



```
ocr_max_image_bytes: int = 4_194_304
```

```
class Config:
    env_file = ".env"
    extra = "ignore"
```

6.3 Metrics

All metrics exposed on `/metrics` (Prometheus format, via `prometheus-fastapi-instrumentator`).

Metric name	Type	Labels	What it answers	PRD §9 NFR
<code>ocr_requests_total</code>	Counter	<code>outcome=</code> <code>{success,no_text,error}</code>	How often does OCR find text vs fail?	—
<code>ocr_latency_seconds</code>	Histogram	<code>strategy=</code> <code>{ocr_only,pipeline,parallel}</code>	Is p99 < 2s per PRD §8.2?	Latency p99 < 2s
<code>ocr_confidence_score</code>	Histogram	—	Distribution of Tesseract word confidence	CER < 15% proxy
<code>ocr_bbox_count</code>	Histogram	—	Are screenshots text-rich?	Debug: content type mix
<code>ocr_image_unreadable_total</code>	Counter	—	How often does OCR produce nothing?	Failure mode tracking
<code>ocr_text_length_chars</code>	Histogram	—	How long is extracted text?	Classifier input sizing

7. NFR Targets & Test Plan

7.1 Latency — p99 < 2s (PRD §8.2)

Target: p99 latency for the full OCR pipeline (preprocess + Tesseract + result) < 2 000 ms on the project hardware (Windows host, RTX 5080, standard HDD).

Test approach:

```
pytest server/tests/test_ocr_latency.py
```

- Load 50 real chat screenshots (WhatsApp + Instagram, various sizes).
- Run `OcrBackend.process()` 3 times each, take the median.

- Assert p99 < 2 000 ms. Assert p50 < 800 ms.
- Parameterize over `strategy={ocr_only, pipeline}`.

Baseline expectation: A 1080p chat screenshot on CPU (no GPU involvement) takes ~200–800ms through preprocessing + Tesseract. The 2s budget is comfortable.

7.2 CER < 15% (PRD §8.2)

Target: Character Error Rate < 15% on real Hebrew chat screenshots.

Test dataset (Meeting 8): - Collect ~50 real WhatsApp/Instagram Hebrew chat screenshots. - Manually transcribe ground-truth text for each. - Run `OcrBackend.process()` on each screenshot. - Compute CER: $(S + D + I) / N$ where S=substitutions, D=deletions, I=insertions, N=ground-truth char count. - Report per-script CER: Hebrew-only words, English-only words, mixed.

How to measure in code:

```
# tools/measure_cer.py
from jiwer import cer # pip install jiwer
score = cer(reference_texts, hypothesis_texts)
assert score < 0.15, f"CER {score:.3f} exceeds 15% target"
```

Fallback plan (PRD §12 risk): If CER > 20%, switch `OcrBackend` to EasyOCR drop-in (same `ImageProcessor` interface, different implementation). No changes to strategy router or classifier needed.

8. Failure Modes & Fallbacks

Failure	Detection	Response	PRD alignment
Tesseract binary not found	<code>TesseractNotFoundError</code> on startup or first call	<code>strategies.py._try_ocr()</code> catches and returns <code>None</code> → <code>Pipeline</code> falls through to <code>VisionBackend</code>	PRD §8.2 "image_unreadable: true"
Missing <code>heb</code> language pack	<code>TesseractError: Failed loading language</code>	Same as above — caught, logged as <code>ocr_tesseract_unavailable</code>	Same
PIL cannot decode image	<code>PIL.UnidentifiedImageError</code>	Propagated as HTTP 422 with <code>{"detail": "image_unreadable"}</code>	PRD §8.2 failure behavior
Image too large (> 4MB)	Checked in <code>OcrBackend.process()</code> before PIL open	Raise <code>ValueError</code> ; Gatekeeper should have caught this first (HTTP 413)	PRD §8.7 Gatekeeper
OCR returns empty string (unreadable image)	<code>bbox_count == 0</code> or <code>extracted_text == ""</code>	Return <code>OcrResult(image_unreadable=True)</code> ; strategy router decides next step	PRD §8.2 fallback
Preprocessing exceeds memory (very large image)	<code>MemoryError</code> during <code>cv2.adaptiveThreshold</code>	Caught, logged at ERROR level, return <code>OcrResult(image_unreadable=True, error="preprocessing_oom")</code>	Graceful degradation

9. Deployment & Config

9.1 Tesseract installation (Windows — project machine)

```
# Install Tesseract with Hebrew language pack
winget install UB-Mannheim.TesseractOCR
# After install, download heb.traineddata + eng.traineddata
# Default install path: C:\Program Files\Tesseract-OCR\
```

Set in `server/.env` :

```
TESSERACT_CMD=C:\Program Files\Tesseract-OCR\tesseract.exe
TESSERACT_LANG=heb+eng
```

9.2 Python dependencies (add to `server/requirements.txt`)

```
pytesseract>=0.3.10
Pillow>=10.0.0
```

```
numpy>=1.24.0
opencv-python-headless>=4.8.0 # headless = no GUI dependency; cv2.adaptiveThreshold
```

Note: `opencv-python-headless` avoids pulling in Qt/GUI libraries that would conflict with headless server deployment.

9.3 WSL2 considerations

When running tests in WSL2 (for training pipeline tests), the Tesseract path is `/usr/bin/tesseract`. Set `TESSERACT_CMD=` (empty) to use PATH. Language pack install: `sudo apt-get install tesseract-ocr-heb tesseract-ocr-eng`.

10. Future Extraction Seam

`OcrBackend` is extraction-ready:

1. **Interface boundary:** depends only on `base.ImageProcessor` (Protocol) and `stdlib`. Zero coupling to FastAPI, Ollama, or the classifier.
 2. **Standalone service path:** wrap in `POST /ocr` FastAPI micro-app; return `OcrResult` JSON. Caller (strategy router) switches from `await ocr.process(bytes)` to `await http_client.post("/ocr", content=bytes)`.
 3. **What needs to change:** add HTTP client in `strategies.py` for the remote-OCR case; add `OcrServiceSettings` with `OCR_SERVICE_URL` env var. The `OcrOnly`, `Pipeline`, and `Parallel` strategy classes remain structurally unchanged — they just call a different `process()` implementation.
 4. **Why defer:** at current scale (local home server, single household), network overhead of a separate OCR process adds ~10–50ms with zero architectural benefit. Extraction pays off only when OCR load needs horizontal scaling (SOM: 5K households simultaneously sending images).
-

11. Open Questions

#	Question	Decision needed by
Q1	Should preprocessing use <code>cv2.adaptiveThreshold</code> (requiring OpenCV) or <code>PIL.ImageFilter.SHARPEN</code> + <code>PIL.ImageOps.autocontrast</code> to avoid the OpenCV dependency? OpenCV is larger but threshold quality is meaningfully better on dark-mode chat screenshots.	Meeting 5 (before first integration test)
Q2	<code>--psm 3</code> (auto) vs <code>--psm 6</code> (uniform block): PSM 3 handles multi-column chat layouts better; PSM 6 is faster. Measure on real screenshots at Meeting 5.	Meeting 5
Q3	Should <code>lang_detected</code> be computed by script detection (Unicode block analysis) or by running Tesseract twice (once per lang) and comparing confidences? Unicode block detection is $O(n)$ and deterministic; dual-run is $\sim 2\times$ slower but more accurate on short words.	Meeting 5
Q4	EasyOCR as the official fallback (PRD §12 risk row "Tesseract OCR poor"): lock the interface contract now so the swap is a 1-file change. Currently <code>OcrBackend</code> and a hypothetical <code>EasyOcrBackend</code> both implement <code>ImageProcessor</code> — confirm this is sufficient.	Meeting 8 if CER > 15%
Q5	Should OCR confidence be exposed in the <code>ImageClassifyResult</code> returned to the client, or kept internal? Currently <code>ImageClassifyResult.confidence</code> is the classifier's confidence, not Tesseract's. Two confidence values may confuse consumers.	Before SDK contract is locked (Meeting 5)